Document Object Model (DOM)

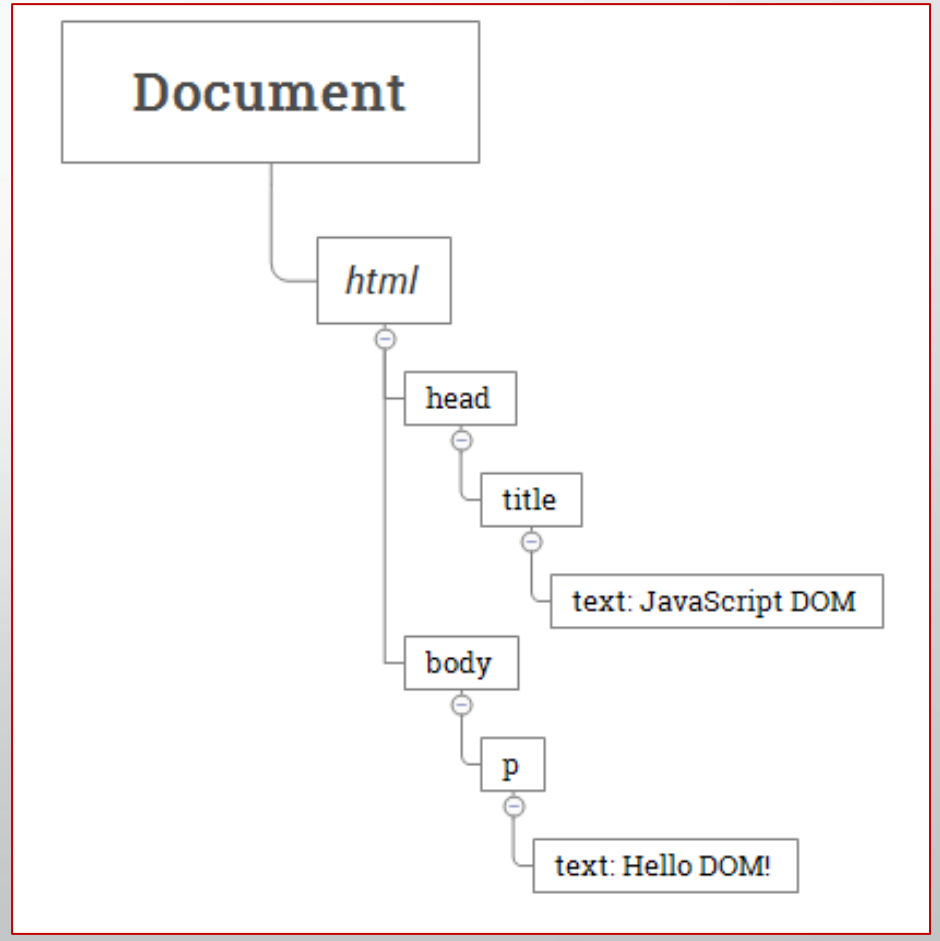
ITI - Assiut Branch

Eng. Sara Ali

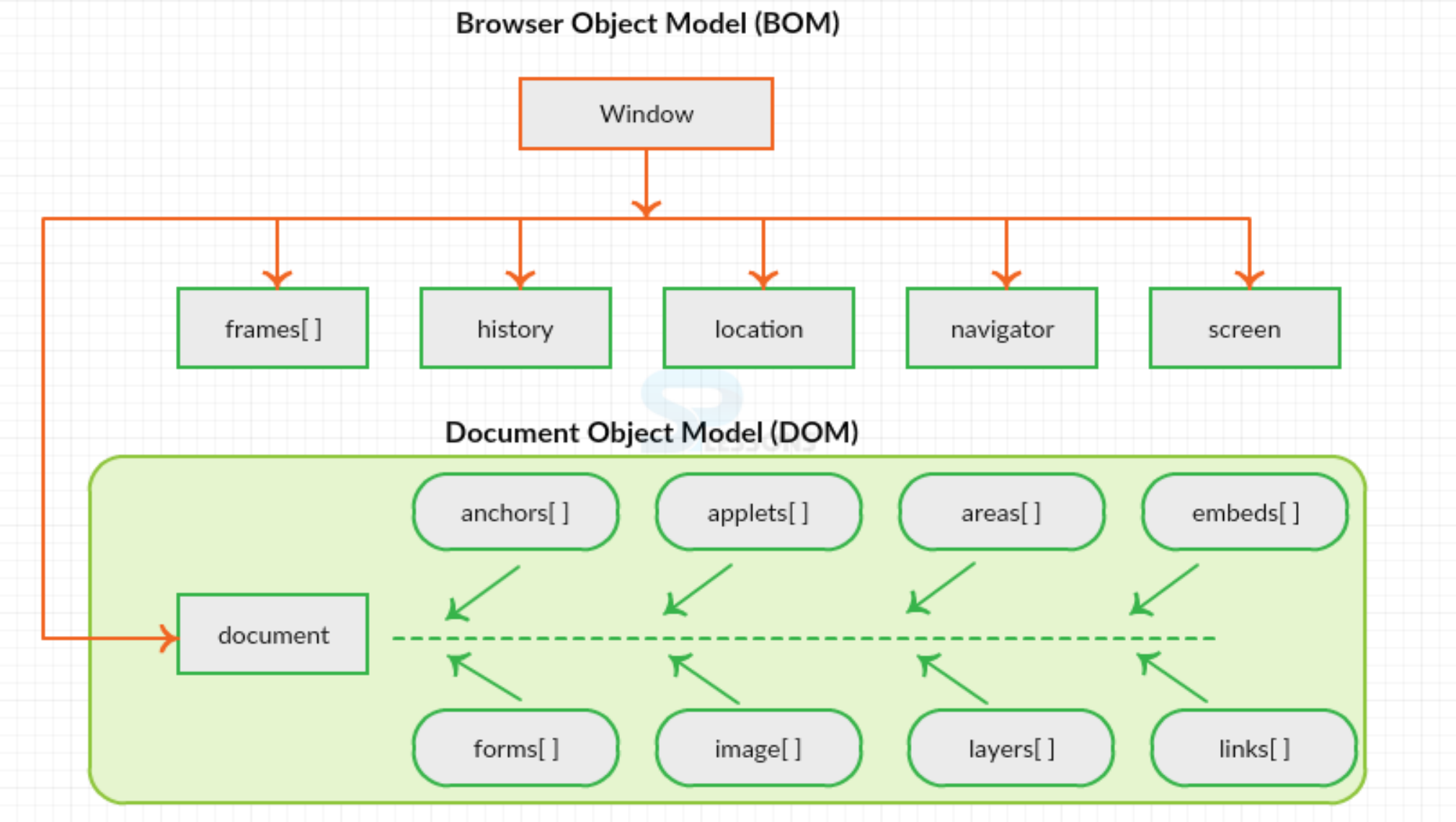
What is DOM?

- The Document Object Model (DOM) represents a document as a tree of nodes. Defines a standard way to access and manipulate HTML documents.

```
<html>
    <head>
        <title>JavaScript DOM</title>
    </head>
    <body>
        <p>Hello DOM!</p>
    </body>
</html>
```



DOM Model


Browser Object Model (BOM)
Window
frames[ ]
history
location
navigator
screen
Document Object Model (DOM)
anchors[ ]
applets[ ]
areas[ ]
embeds[ ]
document
forms[ ]
image[ ]
layers[ ]
links[ ]

Document

- The document object represents the entire HTML document and can be used to access all elements in a page.
- Document Object has its own set of Properties, Collections, Methods & Event handlers.
- With the object model, JavaScript gets all the power it needs to create dynamic HTML:
  - ✔ JavaScript can change all the HTML elements in the page.
  - ✔ JavaScript can change all the HTML attributes in the page.
  - ✔ JavaScript can change all the CSS styles in the page.
  - ✔ JavaScript can remove existing HTML elements and attributes.
  - ✔ JavaScript can add new HTML elements and attributes.
  - ✔ JavaScript can react to all existing HTML events in the page.
  - ✔ JavaScript can create new HTML events in the page.

Document (Cont.)

- **Properties**

Property	Description
bgColor	A string that specifies the background color.
fgColor	A string that specifies the text color.
lastModified	returns the date and time the current document was last modified.
URL	returns the full URL of the current HTML document.
title	A string that specifies the contents of the TITLE tag.

- **Methods**

Method	Description
write()	Writes one or more HTML expressions to a document in the specified window.

Document (Cont.)

- **Collections**

Collection	Description
forms	An array containing an entry for each form in the document
images	An array containing an entry for each image in the document
anchors	An array containing an entry for each anchor in the document.
links	An array containing an entry for each link in the document.

Selecting Elements

1. Select Element By Id
2. Select Elements By Name
3. Select Elements By Tag Name
4. Select Elements By Class Names
5. Select Element By CSS Selectors

Selecting Elements (Cont.)

1. Select Element By Id

- The id is used to uniquely identify an HTML element within the document.
- The getElementById() returns a DOM element specified by an id or null if no matching element found.

```html
<html>
  <head>
   <title>getElementById()Method</title>
  </head>
  <body>
     <p id="message">A paragraph</p>
  </body>
</html>
```

```
const p =
document.getElementById('message');
console.log(p);
//<p id="message">A paragraph</p>
```

Selecting Elements (Cont.)

2. Select Elements By Name

- Every element on an HTML document may have a name. Unlike the id attribute, multiple HTML elements can share the same value of the name attribute.
- The getElementsByName() returns a live NodeList of elements with a specified name.

```
<input type="radio" name="rate"
value="Very poor"> Very poor
<input type="radio" name="rate"
value="Poor"> Poor
<input type="radio" name="rate"
value="OK"> OK
<input type="radio" name="rate"
value="Good"> Good
<input type="radio" name="rate"
value="Very Good"> Very Good
```

```
let rates =
document.getElementsByName('rate');
```

Selecting Elements (Cont.)

3. Select Elements By Tag Name

- The getElementsByTagName() accepts a tag name and returns a list of elements with the matching tag name.

```
<h1>getElementsByTagName() Demo</h1>
<h2>First heading</h2>
<p>This is the first paragraph.</p>
<h2>Second heading</h2>
<p>This is the second paragraph.</p>
<h2>Third heading</h2>
```

```
let headings = document.getElementsByTagName('h2');
Console.log(headings.length); // 3
```

Selecting Elements (Cont.)

4. Select Elements By Class Names

- The getElementsByClassName() method is available on the document object and any HTML element.
- The getElementsByClassName() method accepts a single argument which is a string that contains one or more class names.

```
let btn = document.getElementsByClassName('btn');
```

- If you match elements by multiple classes, you need to use whitespace to separate them like this:

```
let btn = document.getElementsByClassName('btn bt-primary');
```

Selecting Elements (Cont.)

5. Select Elements By CSS Selectors

- Use the JavaScript querySelector() and querySelectorAll() to find elements based on CSS selectors.
- You can call the querySelector() method on the document or any HTML element.
- The querySelector() finds the first element that matches a CSS selector or a group of CSS selectors.
- The querySelectorAll() finds all elements that match a CSS selector or a group of CSS selectors.
- A CSS selector defines elements to which a CSS rule applies.

Selecting Elements (Cont.)

5. Select Elements By CSS Selectors (Cont.)

```
let logo = document.querySelector('#logo'); //finds the first element with the id #logo.

let note = document.querySelector('.menu-item'); //finds the first element with the menu-item class.

let notes = document.querySelectorAll('.menu-item'); //finds all elements with the menu-item class.

let listItems = document.querySelectorAll('ul.nav > li'); //select all li elements that are directly inside a <ul> element with the class nav.

let links = document.querySelectorAll('p a'); //finds all <a> elements inside the p element.
```

Traversing Elements

1. Get the Parent Element
2. Get Siblings of an Element
3. Get Children of an Element

Traversing Elements (Cont.)

1. Get the Parent Element

- To get the parent node of a specified node in the DOM tree, you use the parentNode property. The parentNode is read-only.

```
<div id="main">
    <p class="note">This is a note!</p>
</div>
<script>
        let note = document.querySelector('.note');
        console.log(note.parentNode);
</script>
```

Traversing Elements (Cont.)

2. Get Siblings of an Element

- The nextElementSibling returns the next sibling of an element or null if the element is the last one in the list.

```
let current = document.querySelector('.current');
let nextSibling = current.nextElementSibling;
console.log(nextSibling); //<li>Careers</li>
```

- The previousElementSibling returns the previous sibling of an element or null if the element is the first one in the list.

```
let current = document.querySelector('.current');
let prevSiblings = current.previousElementSibling;
console.log(prevSiblings); //<li>Products</li>
```

```
<ul id="menu">
        <li>Home</li>
        <li>Products</li>
        <li
class="current">Custome
r Support</li>
        <li>Careers</li>
        <li>Investors</li>
        <li>News</li>
        <li>About Us</li>
</ul>
```

Traversing Elements (Cont.)

3. Get Children of an Element

Property	Description
firstElementChild	return the first child Element node.
lastElementChild	return the last child Element node.
children	return all child Element nodes of a specified node.

Manipulating Elements

Property	Description
innerText	returns the human-readable text that takes CSS into account.
innerHTML	get or set HTML contained within the element.

Method	Description
createElement(htmlTag)	accepts an HTML tag name and creates a new HTML element.
element.appendChild()	appends an HTML element to an existing element.

Manging Attributes

Property	Description
element.attributes	access standard and custom attributes of an element.

Method	Description
element.getAttribute(name)	get the attribute value
element.setAttribute(name, value)	set the value for the attribute
element.hasAttribute(name)	check for the existence of an attribute
element.removeAttribute(name)	remove the attribute

Styling

- **Setting inline styles**
  - To set the inline style of an element, you use the style property of that element: element.style

```
element.style.color = 'red';
```

  - If the CSS property contains hyphens (-) for example -webkit-text-stroke you can use the array-like notation ([]) to access the property:

```
element.style.['-webkit-text-stroke'] = 'unset';
```

Styling (Cont.)

CSS	JavaScript
background	background
background-attachment	backgroundAttachment
background-color	backgroundColor
background-image	backgroundImage
background-position	backgroundPosition
background-repeat	backgroundRepeat
border	border
border-bottom	borderBottom
border-bottom-color	borderBottomColor
border-bottom-style	borderBottomStyle
border-bottom-width	borderBottomWidth
border-color	borderColor
border-left	borderLeft

Styling (Cont.)

CSS	JavaScript
border-left-color	borderLeftColor
border-left-style	borderLeftStyle
border-left-width	borderLeftWidth
border-right	borderRight
border-right-color	borderRightColor
border-right-style	borderRightStyle
border-right-width	borderRightWidth
border-style	borderStyle
border-top	borderTop
border-top-color	borderTopColor
border-top-style	borderTopStyle
border-top-width	borderTopWidth
border-width	borderWidth

Styling (Cont.)

CSS	JavaScript
clear	clear
clip	clip
color	color
cursor	cursor
display	display
filter	filter
float	cssFloat
font	font
font-family	fontFamily
font-size	fontSize
font-variant	fontVariant
font-weight	fontWeight
height	height
left	left

Styling (Cont.)

CSS	JavaScript
letter-spacing	letterSpacing
line-height	lineHeight
list-style	listStyle
list-style-image	listStyleImage
list-style-position	listStylePosition
list-style-type	listStyleType
margin	margin
margin-bottom	marginBottom
margin-left	marginLeft
margin-right	marginRight
margin-top	marginTop
overflow	overflow
padding	padding
padding-bottom	paddingBottom

Styling (Cont.)

CSS	JavaScript
padding-left	paddingLeft
padding-right	paddingRight
padding-top	paddingTop
page-break-after	pageBreakAfter
page-break-before	pageBreakBefore
position	position
stroke-dasharray	strokeDasharray
stroke-dashoffset	strokeDashoffset
stroke-width	strokeWidth
text-align	textAlign
text-decoration	textDecoration
text-indent	textIndent
text-transform	textTransform
top	top

Styling (Cont.)

CSS	JavaScript
top	top
vertical-align	verticalAlign
visibility	visibility
width	width
z-index	zIndex

- To completely override the existing inline style, you set the cssText property of the style object. For example:

```
element.style.cssText = 'color:red;backgroundColor:yellow';
```

- Or you can use the setAttribute() method:

```
element.setAttribute('style','color:red;background-color:yellow');
```

Styling (Cont.)

- If you do not want to completely overwrite the existing CSS properties, you can concatenate new CSS property to the cssText as follows:

```
element.style.cssText += 'color:red;backgroundColor:yellow';
```

- The style property returns the inline styles of an element. It doesn’t return the rules that come from elsewhere e.g., styles from an external style sheet.

Styling (Cont.)

- **Get CSS Classes**

- The className is the property of an element. It returns a space-separated list of CSS classes of the element:

```
element.className;
```

- To set a class to an element, you use the following code:

```
element.className += newClassName;
```

- **Manipulate CSS Classes**

- The classList is a read-only property of an element that returns a live collection of CSS classes

```
const classes = element.classList;
```

- Even though the classList is read-only, but you can manipulate the classes it contains using various methods.

Styling (Cont.)

- **Manipulate CSS Classes (cont.)**

```
let div = document.querySelector('#content');

div.classList.add('info');
div.classList.add('info','visible','block');  //add multiple classes

div.classList.remove('visible');
div.classList.remove('block','red'); //remove multiple classes

div.classList.replace('info','warning'); //replace an existing CSS class with a new one

div.classList.contains('warning'); // true //check if the element has a specified class

div.classList.toggle('visible');
```

Image

- **HTML**

```
<img name="img1" id="img1" src="img1.jpg">
```

- Any images created in a document are then stored in an array in the document.images property.
- To reference the <form> element, you can use DOM selecting methods such as getElementById():

```
document.getElementById("img1");
document.images[0];
```

Image (Cont.)

- **Properties:**

name	id	src	height	width

- **Event handlers:**

onmouseout	onmouseover	onmousemove	onclick	ondblclick

Form

- To create a form in HTML, you use the <form> element:

```
<form action="/signup" method="post" id="signup">
</form>
```

- To reference the <form> element, you can use DOM selecting methods such as getElementById():

```
const form = document.getElementById('subscribe');
```

- An HTML document can have multiple forms. The document.forms property returns a collection of forms. To reference a form, you use an index.

```
const form = document.forms[0];
```

Form (Cont.)

- **Submitting a form**
  - When you submit the form, the submit event is fired before the request is sent to the server. This gives you a chance to validate the form data. If the form data is invalid, you can stop submitting the form.
  - To attach an event listener to the submit event, you use the addEventListener() method or use onsubmit eventhandler

```
const form = document.getElementById('signup');
form.addEventListener('submit', (event) => {
        // handle the form data
        event.preventDefault();
});
```

```
<form
id="“signup” onsubmit="myFunction()
">
    <input type="submit">
</form>
```

Form (Cont.)

- **Properties**

Property	Description
elements[]	An array containing all of the elements of the form. Use it to loop through form easily.
length	The number of elements in the form.
name	The name of the form.
id	The id of the form.

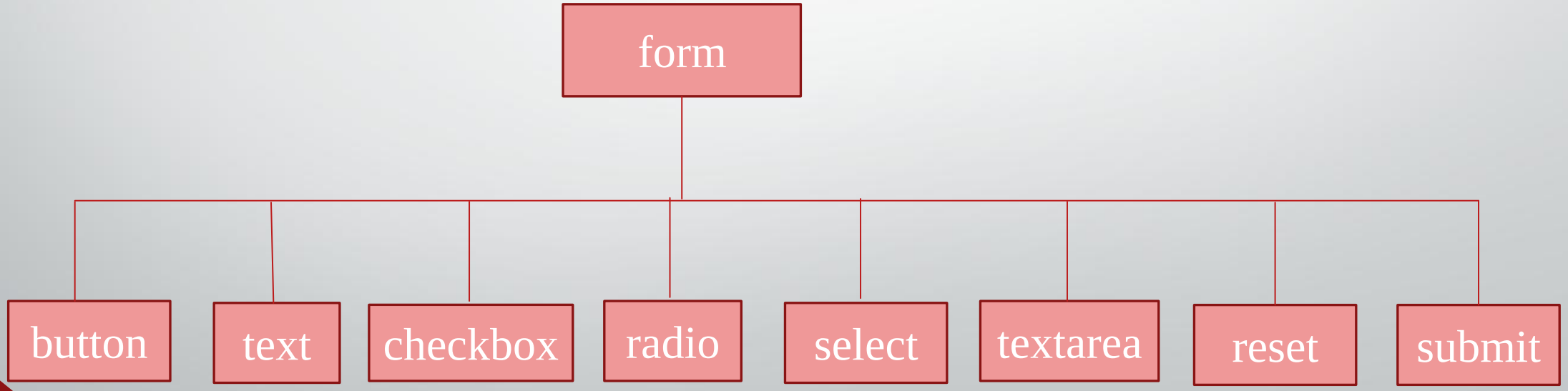
- **Methods**

Method	Description
reset()	Resets the form. Clicking the reset button clears all contents that the user has made.
submit()	Submits a form. Clicking the submit button submits the content of the form to the server

Form (Cont.)

- **Event handlers:**

Event	Description
onReset	Code is executed when the form is reset (by clicking on "reset" button)
onSubmit	Code is executed when form is submitted



Button

- **HTML**

```
<input type="button“ id="id“  value="string"/>
```

- **Properties**

properties	Description
value	Sets or returns the text that is displayed on the button
disabled	Sets or returns whether or not a button should be disabled

- **Methods**

Method	Description
blur()	Removes focus from the field.
focus()	Assigns focus to the field; places the cursor in the control.
click()	Simulates a mouse-click on a button

Button (cont.)

- **Event Handlers**

Event Handler	Event
onclick	The mouse is clicked and released with the cursor positioned over the button.
ondblclick	The mouse is double-clicked with the cursor positioned over the button.
onmousedown	The mouse is pressed down with the cursor positioned over the button.
onmouseout	The mouse cursor is moved off the button.
onmouseover	The mouse cursor is moved on top of the button.
onmouseup	The mouse button is released with the cursor positioned over the button.

Text

- **HTML**

```
<input type=“text“ id="id“  value="string“
maxlength="n"  size=“x"
/>
```

- **Properties**

properties	Description
value	Sets or returns the value of the value attribute of a text field.
readOnly	Sets or returns whether or not a text field should be read-only.
disabled	Sets or returns whether or not a text field should be disabled

Text (cont.)

- **Methods**

Method	Description
blur()	Removes focus from the field.
focus()	Assigns focus to the field; places the cursor in the control.
select()	Selects, or highlights, the content of the control.

- **Event Handlers**

Event Handler	Description
onfocus	The field gains focus when the user tabs into or clicks inside the control.
onblur	The field loses focus when the user tabs from or clicks outside the control.
onchange	The field loses focus after the contents of the control have changed.

Checkbox

- **HTML**

```
<input type=“checkbox“ id="id“  value="string“ checked/>
```

- **Properties**

properties	Description
checked	A true or false value indicating whether a button is checked.
value	Sets or returns the value of the value attribute of the checkbox.

- **Methods**

Method	Description
blur()	Removes focus from the field.
focus()	Assigns focus to the field; places the cursor in the control.
click()	Simulates a mouse-click on a radio button

Checkbox (cont.)

- **Event Handlers**

Event Handler	Description
onfocus	The control gains focus.
onblur	The control loses focus.
onclick	The button is clicked.

Radio button

- **HTML**

```
<input type=“radio“ id="id“  value="string“ checked/>
```

- **Properties**

properties	Description
length	The number of radio buttons with the same name.
checked	A true or false value indicating whether a button is checked.
value	Sets or returns the value of the value attribute of the radio button

Radio button (cont.)

- **Methods**

Method	Description
blur()	Removes focus from the field.
focus()	Assigns focus to the field; places the cursor in the control.
click()	Simulates a mouse-click on a radio button.

- **Event Handlers**

Event Handler	Description
onfocus	The control gains focus.
onblur	The control loses focus.
onclick	The button is clicked.

Select

- **HTML**

```
<select id="id“ multiple size="n">
            <option value="string" selected >  label </option>
          ...
</select>
```

- **Properties**

Property	Description
length	The number of options in the list.
selectedIndex	The index number, beginning with 0, of the selected option.
options[]	An array of the options in the list. Used to reference properties associated with the options; e.g., options[1].value or options[2].text.
value	Sets or returns the value of the selected option in a drop-down list.
text	The text label associated with an option.
multiple	Sets or returns whether or not multiple items can be selected.

Select (cont.)

- **Properties**

Property	Description
selected	A true or false value indicating whether an option is chosen.
index	Sets or returns the index position of an option

- **Methods**

Method	Description
add()	Adds an option to a dropdown list
remove()	Removes an option from a dropdown list
focus()	Assigns focus to the field; places the cursor in the control.
blur()	Removes focus from a dropdown list

Select (cont.)

- **Event Handlers:**

Event Handler	Description
onfocus	The control gains focus.
onblur	The control loses focus.
onchange	A different option from the one currently displayed is chosen.

Introducing SPAN & DIV

- Used to add text or HTML elements to the page without removing the page contents on the run time.

- HTML

```
<span id="s1">
        Hello There!
</ span>
```

```
<div id="d1">
        Hello There!
</div>
```

Introducing SPAN & DIV (cont.)

```
<script>
        document.getElementById("d1").textContent="new text"
        //new text
        document.getElementById("d1").textContent="<b>new text</b>";
        //<b>new text</b>
        document.getElementById("d1").innerHTML="<b>new text</b>";
        //new text
</script>
```

This object

- Use this keyword to refer to the current object.
- Self reference to the object is used :

```
<input type=“text” onFocus = "this.value=‘You are in!’“/>
```

- Passing current Object as a function parameter:

```
function myFunction(myObject)
{
        myObject.value = ”In the function!!“
}

<input type=“text” onClick=“myFunction(this)”/>
```

THANK YOU